

Technical University of Cluj-Napoca

MATLAB
Based Design and Study of
Planetary Motion

Student

Varga Radu
GR-2022

Coordinator

Conf.dr.ing. Cirlugea Mihaela

Table of contents:

1. Introduction

2. Theoretical Presentation

- Mathematical Formulas and Principles
- Conceptual Framework of the Simulation

3. Experimental Part

- Setup of the MATLAB Environment
- Implementation of Solar System Simulation
- Design of Graphical User Interface (GUI)
- Code Explanation

4. Conclusion

5. References

6. Appendix

- Programming Code

1. Introduction

Historic Presentation of MATLAB

MATLAB, short for Matrix Laboratory, was originally developed in the late 1970s by Cleve Moler, a professor of computer science at the University of New Mexico. Initially, it served as a teaching tool for numerical computations, giving students access to the LINPACK and EISPACK libraries for solving linear algebra problems. In 1984, MATLAB's development took a major leap when Moler co-founded MathWorks with Jack Little and Steve Bangert. The software was commercialized, and its first version, MATLAB 1.0, was launched for Sun Microsystems and DEC VAX systems. This marked its transition from an academic teaching aid to a professional software environment.

Since its commercialization, MATLAB has undergone significant enhancements and broadened its scope. From its original focus on matrix computations, it has evolved into a versatile platform for diverse applications, including signal processing, machine learning, control systems, and image analysis. It has become indispensable in engineering, scientific research, and education. Widely recognized for its intuitive syntax, robust functionality, and extensive documentation, MATLAB is a preferred tool in both academia and industry. Its large user community and the continuous introduction of specialized toolboxes have further solidified its status as a leading computational software.

1. Introduction

The **Planetary Motion** project is designed to explore the dynamics of planetary motion through a visual and interactive MATLAB application. This project allows users to delve into concepts related to astronomy, physics, and mathematics by simulating the orbital and rotational behavior of planets. By integrating formulas such as Kepler's laws of planetary motion and sinusoidal

functions for rotational speeds, the project provides an engaging way to understand the relationships between planetary distances, orbital speeds, and sizes.

The motivation behind this project stems from the importance of blending theoretical knowledge with practical implementation. MATLAB, a versatile platform for mathematical and engineering computations, was chosen for its ability to handle complex calculations and create dynamic visualizations as well because it is the program studied at this time. The project also emphasizes user interaction, enabling changes to planetary parameters such as size, speed, and distance, which immediately reflect in graphs and 3D representations.

Ultimately, this project bridges scientific theory with computational tools, offering a valuable learning experience for understanding planetary motion.

2. Theoretical Presentation

Kepler's Laws of Planetary Motion

1. Elliptical Orbits:

Kepler's First Law states that planets move in elliptical orbits, with the Sun located at one of the foci. This principle provides the foundational geometry for planetary motion, explaining why planets are not equidistant from the Sun during their orbits. In this project, circular approximations are used for simplicity, aligning with the mathematical functions available in MATLAB.

2. Equal Areas in Equal Times:

According to Kepler's Second Law, the orbital speed of a planet changes depending on its distance from the Sun. When the planet is closer to the Sun (perihelion), it moves faster, and when it is farther away (aphelion), it slows down. This change in velocity was simulated in the project through sinusoidal graphs to approximate variations in orbital speed over time.

3. Harmonic Relationship:

Kepler's Third Law establishes a relationship between the orbital period of a planet and its average distance from the Sun. Mathematically:

$$T^2 \approx a^3$$

where T is the orbital period, and a is the semi-major axis of the ellipse. This relationship is crucial for calculating orbital speeds and distances for the project.

Newton's law of universal gravitation provides further depth, stating:

$$F = \frac{G \cdot m_1 \cdot m_2}{r^2}$$

Here, F represents the gravitational force between two masses (m_1 and m_2), r is the distance between their centers, and G is the gravitational constant. This relationship quantifies the attractive force exerted by the Sun on a planet, keeping it in a stable orbit. When combined with the centripetal force equation:

$$F = \frac{mv^2}{r}$$

we derive expressions for orbital speed (v) as a function of distance and mass, which are critical for simulating dynamic planetary systems.

Conceptual Framework

Leveraging these mathematical principles, the simulation incorporates MATLAB's computational and visualization capabilities to create an interactive environment for exploring planetary motion. The design integrates multiple components:

1. **Interactive Controls:** Sliders and input fields allow users to modify planetary properties such as diameter, orbital speed, and distance from the Sun. These changes directly update the simulation, reinforcing the mathematical relationships.
2. **Rotational Dynamics:** Sinusoidal signals are used to represent the rotational speeds of planets over time. The equation for these signals, derived as:

$$y(t) = A \sin\left(2\pi \frac{t}{T}\right)$$

where A is the amplitude, t is time, and T is the orbital period, demonstrates the periodic nature of planetary rotation. This allows the simulation to visualize changes in speed or distance dynamically.

3. **Orbital Speed and Distance Graphs:** The simulation includes visualizations of orbital speed versus distance from the Sun. These graphs are based on the formula:

$$v = \sqrt{\frac{G \cdot M}{r}}$$

where M is the mass of the Sun. The inverse square root dependency highlights how orbital speeds decrease with increasing distance from the Sun, illustrating the balance of gravitational and centripetal forces.

4. **Textured 3D Planets:** The planets are modeled as spheres with real-world textures, enhancing the simulation's visual fidelity.

5. **Gravitational Force Representation:** Using MATLAB's graphical capabilities, the gravitational forces between the Sun and planets are modeled with vectors or lines.

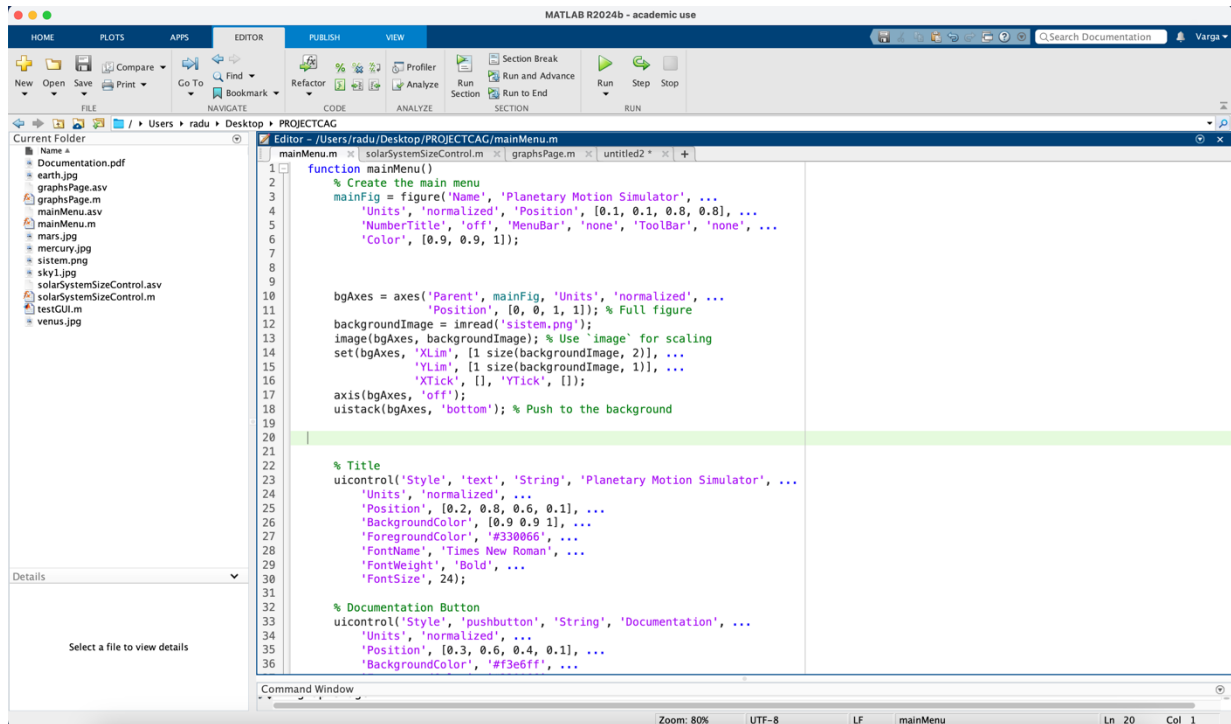
3. Experimental part

The experimental phase of the project focused on applying theoretical principles to create an interactive and visually engaging MATLAB simulation of planetary motion. This section outlines the setup, implementation, and design process.

Setup of the MATLAB Environment

MATLAB was configured with the necessary toolboxes, such as the Image Processing Toolbox, to handle 3D graphics, textures, and interactive controls. Assets like planetary textures and background images were organized into a project folder to ensure smooth integration. Functions like `sphere`, `surface`, and

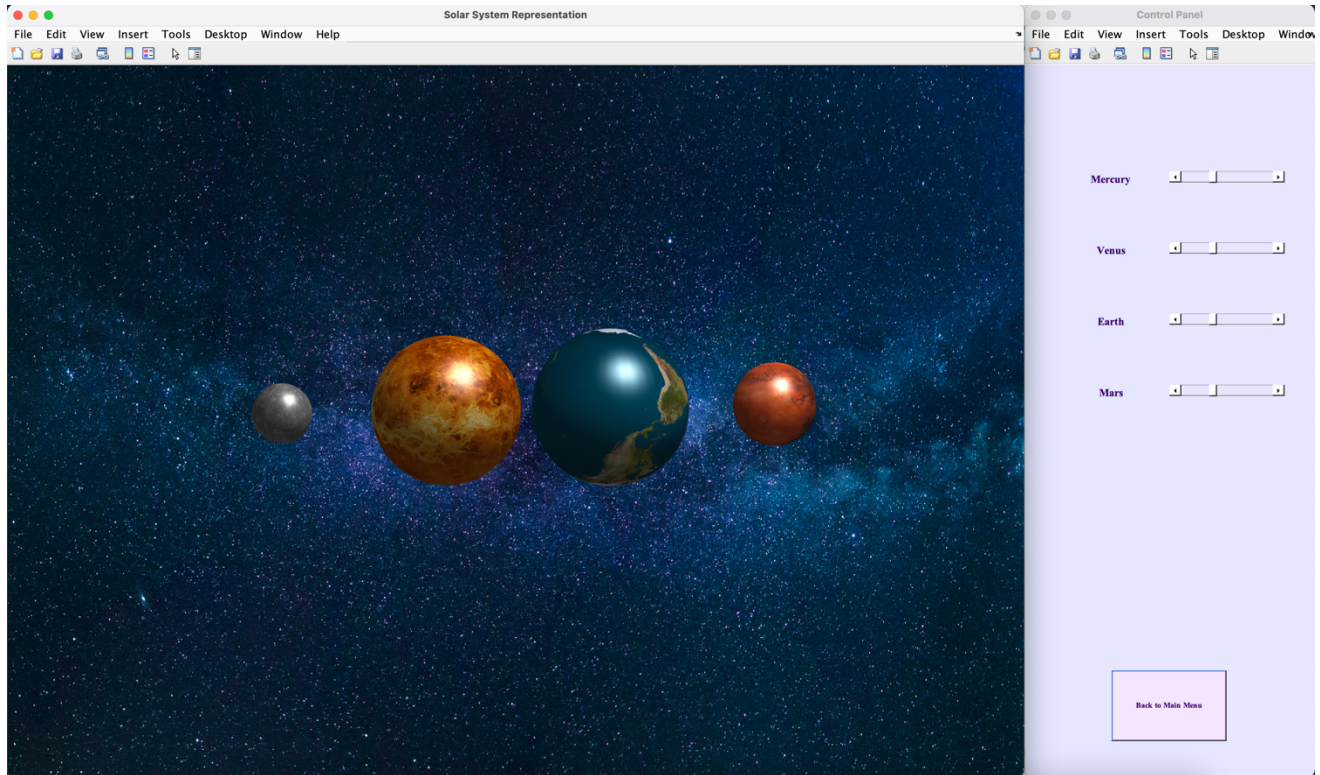
uicontrol were used to build the foundational elements of the simulation, while debugging tools were employed to refine functionality.



Implementation of Solar System Simulation

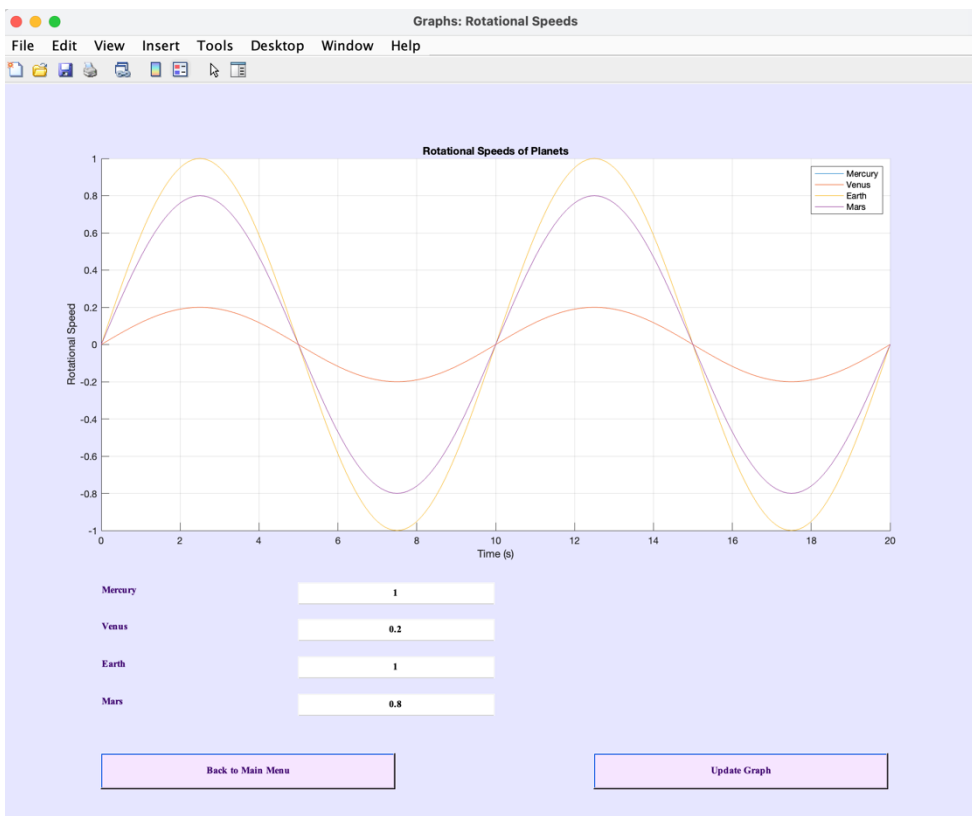
1. 3D Planetary Models:

Planets were represented as textured 3D spheres with realistic scaled radii and distances. Textures, such as those for Earth, Mars, Venus, and Mercury, were applied using the CData property. Lighting effects (e.g., lighting phong) added depth to the visual representation.



2. Planetary Motion:

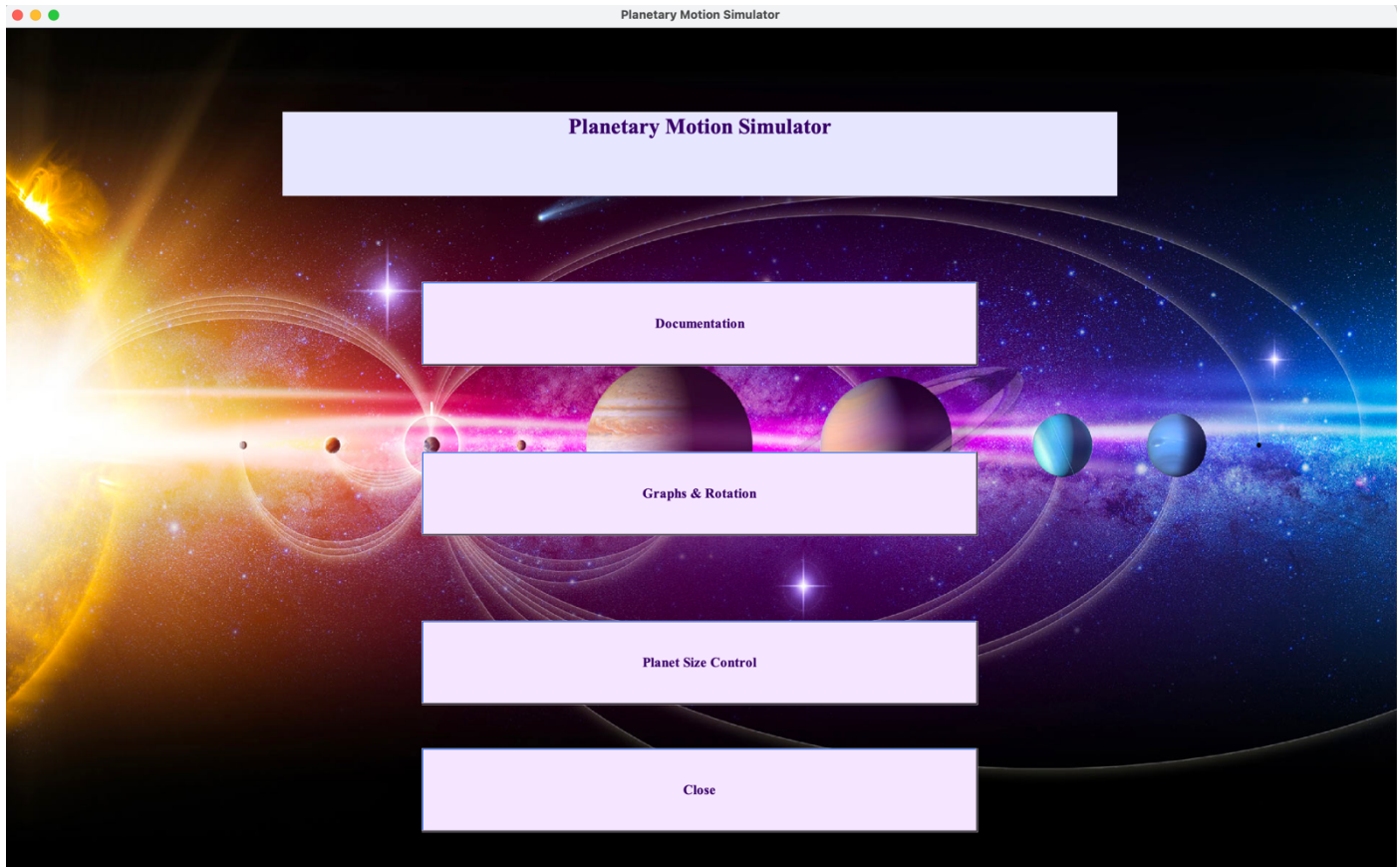
Orbital speeds were calculated using:



$$v = \sqrt{\frac{G \cdot M}{r}}$$

where G is the gravitational constant, M the Sun's mass, and r the distance from the Sun. The planets were animated to simulate orbital motion, showcasing their varying velocities at different distances from the Sun.

Main Menu: The central hub of the application, featuring buttons to access the simulation, graphs, and documentation. The menu utilized a visually engaging background image to enhance user experience.



Code Explanation:

Applying Textures to Planets

To make the planets more realistic, textures representing Mercury, Venus, Earth, and Mars were applied to 3D spheres. This required using MATLAB's `surface` function along with the `CData` property.

```
%initialize planetele
numPlanets = size(planets, 1);
planetHandles = gobjects(numPlanets, 1);

for i = 1:numPlanets
    [x, y, z] = sphere(50);
    scaledRadius = planets{i, 2} / 10000; % Scale size for visualization

    % Apply texture to the planet
    texture = imread(planets{i, 4}); %pun texture pe ele
    planetHandles(i) = surface(scaledRadius * x + 3 * i, ...
                              scaledRadius * y, ...
                              scaledRadius * z, ...
                              'FaceColor', 'texturemap', ...
                              'EdgeColor', 'none', ...
                              'CData', texture); % Apply texture
end

lighting phong;
camlight;
```

Real-Time Graph Updates with User Input

A unique feature of the project was the dynamic update of graphs based on user-modified values for planetary properties. Sliders and input fields were linked to the graph data.

```
% Callback: Update Graph
function updateGraph(~, ~)
    % New rotational speeds - input fields
    for j = 1:length(planets)
        newSpeed = str2double(get(inputHandles(j), 'String')); % Convert input to number
        planets{j, 2} = newSpeed; % Update viteza planetei
    end

    % Update graph-new values
    for j = 1:length(planets)
        newSignal = planets{j, 2} * sin(2 * pi * t / 10);
        set(plotHandles(j), 'YData', newSignal); % Update YData of the plot
    end
end
```

- The updateGraph function retrieves user input for new planetary speeds using str2double().
- These values dynamically update the graph's YData property, reflecting the changes in real-time.
- Challenges: Managing synchronization between sliders, input fields, and the graph while ensuring valid input.

4. Conclusion

The project, *Design and Study of Planetary Motion*, successfully combined mathematical modeling, simulation techniques, and interactive visualization in MATLAB to create a comprehensive representation of planetary motion. By integrating Kepler's laws of planetary motion, gravitational force calculations, and interactive features, the project provided an engaging and educational tool for studying celestial mechanics.

The implementation involved several challenging aspects, such as applying textures to 3D spheres, dynamically updating graphs based on user input, and designing an intuitive graphical user interface. Each of these components required careful planning and execution to ensure functionality, accuracy, and user-friendliness.

In conclusion, this project achieved its primary objective of simulating and visualizing planetary motion while providing valuable insights into the underlying principles. Future enhancements could include the incorporation of additional celestial bodies, more advanced orbital mechanics, or even real-time planetary data integration. This would expand the scope and applicability of the simulation, making it an even more versatile educational tool.

5. References

- <https://mathworks.com/help/matlab/>
- <https://www.solarsystemscope.com/textures/>
- <https://wallpapers.com/blue-galaxy>
- <https://ch.mathworks.com/products/matlab.html>

6. Appendix

%%
mainMenu.m

```
function mainMenu()
% Create the main menu
mainFig = figure('Name', 'Planetary Motion Simulator', ...
    'Units', 'normalized', 'Position', [0.1, 0.1, 0.8, 0.8], ...
    'NumberTitle', 'off', 'MenuBar', 'none', 'ToolBar', 'none', ...
    'Color', [0.9, 0.9, 1]);

bgAxes = axes('Parent', mainFig, 'Units', 'normalized', ...
    'Position', [0, 0, 1, 1]); % Full figure
backgroundImage = imread('sistem.png');
image(bgAxes, backgroundImage); % Use `image` for scaling
set(bgAxes, 'XLim', [1 size(backgroundImage, 2)], ...
    'YLim', [1 size(backgroundImage, 1)], ...
    'XTick', [], 'YTick', []);
axis(bgAxes, 'off');
uistack(bgAxes, 'bottom'); % Push to the background
```

```

% Title
uicontrol('Style','text','String','Planetary Motion Simulator', ...
    'Units','normalized', ...
    'Position',[0.2, 0.8, 0.6, 0.1], ...
    'BackgroundColor',[0.9 0.9 1], ...
    'ForegroundColor','#330066', ...
    'FontName','Times New Roman', ...
    'FontWeight','Bold', ...
    'FontSize', 24);

% Documentation Button
uicontrol('Style','pushbutton','String','Documentation', ...
    'Units','normalized', ...
    'Position',[0.3, 0.6, 0.4, 0.1], ...
    'BackgroundColor','#f3e6ff', ...
    'ForegroundColor','#330066', ...
    'FontName','Times New Roman', ...
    'FontWeight','Bold','FontSize',15, ...
    'Callback', @openDocumentation);

% Graphs & Rotation Button
uicontrol('Style','pushbutton','String','Graphs & Rotation', ...
    'Units','normalized', ...
    'Position',[0.3, 0.4, 0.4, 0.1], ...
    'BackgroundColor','#f3e6ff', ...
    'ForegroundColor','#330066', ...
    'FontName','Times New Roman', ...
    'FontWeight','Bold','FontSize',15, ...
    'Callback', @openGraphsPage);

% Size Control Button
uicontrol('Style','pushbutton','String','Planet Size Control', ...
    'Units','normalized', ...
    'Position',[0.3, 0.2, 0.4, 0.1], ...
    'BackgroundColor','#f3e6ff', ...
    'ForegroundColor','#330066', ...
    'FontName','Times New Roman', ...
    'FontWeight','Bold','FontSize',15, ...
    'Callback', @openSizeControl);

% Close Button
uicontrol('Style','pushbutton','String','Close', ...
    'Units','normalized', ...
    'Position',[0.3, 0.05, 0.4, 0.1], ...
    'BackgroundColor','#f3e6ff', ...
    'ForegroundColor','#330066', ...
    'FontName','Times New Roman', ...
    'FontWeight','Bold','FontSize',15, ...
    'Callback', @(~, ~) close(mainFig));

% Callback Open Documentation
function openDocumentation(~, ~)
    docPath = 'Documentation.pdf';

```

```

        if isfile(docPath)
            open(docPath);
        else
            error('Documentation file not found.', 'File Error');
        end
    end
end

% Callback- Open Graphs Page
function openGraphsPage(~, ~)
    close(mainFig);
    graphsPage();
end

% Callback-open Size Control Page
function openSizeControl(~, ~)
    close(mainFig);
    solarSystemSizeControl();
end
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
solarSystemSizeControl.m

function solarSystemSizeControl()

% Planets nume, diametru initial (km), Color, Textura
planets = {
    'Mercury', 4879, [0.7 0.7 0.7], 'mercury.jpg';
    'Venus', 12104, [0.9 0.8 0.5], 'venus.jpg';
    'Earth', 12742, [0.4 0.8 1.0], 'earth.jpg';
    'Mars', 6779, [1.0 0.5 0.5], 'mars.jpg';
};

% Create the 3D representation figure
solarFig = figure('Name', 'Solar System Representation', ...
    'Units', 'normalized', 'Position', [0.2, 0.2, 0.6, 0.7], ...
    'NumberTitle', 'off', 'Color', [0.0 0.0 0.2]);
hold on;
axis equal;
axis off;
view(3);

bgAxes = axes('Parent', solarFig, 'Units', 'normalized', ...
    'Position', [0, 0, 1, 1]); % Cover the entire figure
backgroundImage = imread('sky1.jpg'); % Replace with sky1
imagesc(backgroundImage, 'Parent', bgAxes); % Display img
set(bgAxes, 'YDir', 'reverse'); % Flip Y-axis sa nu apara invers
set(bgAxes, 'HandleVisibility', 'off'); % No background interaction
uistack(bgAxes, 'bottom'); %background axes to the back

```

```

%initializez planetele
numPlanets = size(planets, 1);
planetHandles = gobjects(numPlanets, 1);

for i = 1:numPlanets
    [x, y, z] = sphere(50);
    scaledRadius = planets{i, 2} / 10000; % Scale size for
visualization

    % Apply texture to the planet
    texture = imread(planets{i, 4}); %pun textura pe ele
    planetHandles(i) = surface(scaledRadius * x + 3 * i, ...
        scaledRadius * y, ...
        scaledRadius * z, ...
        'FaceColor', 'texturemap', ...
        'EdgeColor', 'none', ...
        'CData', texture); % Apply texture

end

lighting phong;
camlight;

%Putting the same color as the main menu

controlFig = figure('Name', 'Control Panel', ...
    'Units', 'normalized', 'Position', [0.8, 0.3, 0.2, 0.7], ...
    'NumberTitle', 'off', 'Color', [0.9 0.9 1]);

% Add sliders and labels for each planet
for i = 1:numPlanets
    % Label for the planet
    uicontrol('Style', 'text', 'String', planets{i, 1}, ...
        'Units', 'normalized', ...
        'Position', [0.1, 0.9 - 0.1 * i, 0.4, 0.05], ...
        'BackgroundColor', [0.9 0.9 1], ...
        'ForegroundColor', '#330066', ...
        'FontName', 'Times New Roman', ...
        'FontSize', 14, 'FontWeight', 'bold');

    % Slider for adjusting size
    uicontrol('Style', 'slider', ...
        'Units', 'normalized', ...
        'Position', [0.5, 0.9 - 0.1 * i, 0.4, 0.05], ...
        'Min', planets{i, 2} * 0.5, 'Max', planets{i, 2} * 2, ...
        'Value', planets{i, 2}, ...
        'BackgroundColor', [0.9 0.9 1], ...
        'Callback', @(src, ~) updateSize(i, src.Value));

end

% Back to Main Menu Button
uicontrol('Style', 'pushbutton', 'String', 'Back to Main Menu', ...
    'Units', 'normalized', ...
    'Position', [0.3, 0.05, 0.4, 0.1], ...
    'BackgroundColor', '#f3e6ff', ...
    'ForegroundColor', '#330066', ...
    'FontName', 'Times New Roman', ...

```

```

        'FontWeight', 'Bold', ...
        'Callback', @backToMainMenu);

% Callback ptUpdate Planet Size
function updateSize(index, newDiameter)

    % Update planet size in the 3D visualization
    scaledRadius = newDiameter / 10000;
    [x, y, z] = sphere(50);
    set(planetHandles(index), ...
        'XData', scaledRadius * x + 3 * index, ...
        'YData', scaledRadius * y, ...
        'ZData', scaledRadius * z);
    fprintf('%s: Diameter updated to %.2f km\n', planets{index, 1},
newDiameter);
end

% Callback Back to Main Menu
function backToMainMenu(~, ~)

    if isvalid(controlFig)
        close(controlFig);
    end
    if isvalid(solarFig)
        close(solarFig);
    end
    mainMenu();
end
end

```

%%%

graphsPage.m

```

function graphsPage()

% Planets ,nume si viteza de rotatie
planets = {
    'Mercury', 1;
    'Venus', 0.2;
    'Earth', 1;
    'Mars', 0.8;
};

% vectorul de timp
t = 0:0.1:20; % 20s

graphFig = figure('Name', 'Graphs: Rotational Speeds', ...
    'Units', 'normalized', 'Position', [0.2, 0.2, 0.6, 0.7], ...
    'NumberTitle', 'off', 'Color', [0.9 0.9 1]);

% Axes for plotting
axesHandle = axes('Position', [0.1, 0.4, 0.8, 0.5]);
hold on;

```

```

% Initial plot handles for each planet
plotHandles = gobjects(length(planets), 1);
for i = 1:length(planets)
    rotationalSpeed = planets{i, 2};
    signal = rotationalSpeed * sin(2 * pi * t / 10);
    plotHandles(i) = plot(t, signal, 'DisplayName', planets{i, 1});
end
hold off;

```

```

legend show;
title('Rotational Speeds of Planets');
xlabel('Time (s)');
ylabel('Rotational Speed');
grid on;

```

```

% user input fields
inputHandles = gobjects(length(planets), 1);
for i = 1:length(planets)

    % labels for input
    uicontrol('Style', 'text', 'String', planets{i, 1}, ...
        'Units', 'normalized', ...
        'Position', [0.1, 0.3 - (i - 1) * 0.05, 0.2, 0.03], ...
        'BackgroundColor', [0.9 0.9 1], ...
        'ForegroundColor', '#330066', ...
        'FontName', 'Times New Roman', ...
        'FontWeight', 'Bold', ...
        'HorizontalAlignment', 'left');

    % input for rotational speed.
    inputHandles(i) = uicontrol('Style', 'edit', ...
        'String', num2str(planets{i, 2}), ... %val initiala
        'Units', 'normalized', ...
        'Position', [0.3, 0.3 - (i - 1) * 0.05, 0.2, 0.03], ...
        'BackgroundColor', '#ffffff', ...
        'ForegroundColor', '#000000', ...
        'FontName', 'Times New Roman', ...
        'FontWeight', 'Bold', ...
        'HorizontalAlignment', 'center');

end

% Update Graph-button
uicontrol('Style', 'pushbutton', 'String', 'Update Graph', ...
    'Units', 'normalized', ...
    'Position', [0.6, 0.05, 0.3, 0.05], ...
    'BackgroundColor', '#f3e6ff', ...
    'ForegroundColor', '#330066', ...
    'FontName', 'Times New Roman', ...
    'FontWeight', 'Bold', ...
    'Callback', @updateGraph);

uicontrol('Style', 'pushbutton', 'String', 'Back to Main Menu', ...
    'Units', 'normalized', ...

```

```

        'Position', [0.1, 0.05, 0.3, 0.05], ...
        'BackgroundColor', '#f3e6ff', ...
        'ForegroundColor', '#330066', ...
        'FontName', 'Times New Roman', ...
        'FontWeight', 'Bold', ...
        'Callback', @backToMain);

    % Callback: Update Graph
    function updateGraph(~, ~)
        % New rotational speeds - input fields
        for j = 1:length(planets)
            newSpeed = str2double(get(handles(j), 'String')); % Convert
input to number
            planets{j, 2} = newSpeed; % Update viteza planetei
        end

        % Update graph-new values
        for j = 1:length(planets)
            newSignal = planets{j, 2} * sin(2 * pi * t / 10);
            set(handles(j), 'YData', newSignal); % Update YData of the plot
        end
    end

    % Callback Back-Main Menu
    function backToMain(~, ~)
        close(graphFig);
        mainMenu();
    end
end

```